

DATE: 5/01/2026
EX NO :2

NAME: Faatina S
REG NO :3122 23 5001 046

UCS2612 & Machine Learning Laboratory

Experiment 2: Loan Amount Prediction using Linear Regression

Aim

To apply the Linear Regression algorithm to predict the sanctioned loan amount for users using historical loan data and to evaluate the model performance through statistical metrics and visualizations

Libraries Used

- **Pandas** – for loading and manipulating the dataset
- **NumPy** – for numerical and matrix operations
- **Matplotlib** – for plotting and visualization
- **Seaborn** – for correlation heatmap and EDA visuals
- **Scikit-learn** – for model building, scaling, splitting, and evaluation

Objective

- To Predict loan sanction amount using Linear Regression
- To Perform data preprocessing including encoding and normalization
- To Analyze relationships among features using EDA
- To Evaluate the model using MAE, MSE, RMSE, and R²
- To Visualize actual vs predicted values and residual behavior

Mathematical Description

Linear Regression assumes a linear relationship between feature vector X and target y .

Model

$$\hat{y} = w_0 + \sum_{j=1}^p w_j x_j$$

Cost Function

$$J(w) = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Closed Form Solution

$$w = (X^T X)^{-1} X^T y$$

Evaluation

- $MAE = \frac{1}{n} \sum |y - \hat{y}|$
- $MSE = \frac{1}{n} \sum (y - \hat{y})^2$
- $RMSE = \sqrt{MSE}$
- $R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$
- $\text{Adjusted } R^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$

#LOAD THE DATASET

```
import pandas as pd
df=pd.read_csv("train.csv")
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 24 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Customer ID                          30000 non-null  object
 1   Name                                30000 non-null  object
 2   Gender                              29947 non-null  object
 3   Age                                 30000 non-null  int64
 4   Income (USD)                        25424 non-null  float64
 5   Income Stability                     28317 non-null  object
 6   Profession                           30000 non-null  object
 7   Type of Employment                  22730 non-null  object
 8   Location                            30000 non-null  object
 9   Loan Amount Request (USD)           30000 non-null  float64
10   Current Loan Expenses (USD)         29828 non-null  float64
11   Expense Type 1                      30000 non-null  object
12   Expense Type 2                      30000 non-null  object
13   Dependents                          27507 non-null  float64
14   Credit Score                        28297 non-null  float64
15   No. of Defaults                     30000 non-null  int64
16   Has Active Credit Card               28434 non-null  object
17   Property ID                         30000 non-null  int64
18   Property Age                        25150 non-null  float64
19   Property Type                       30000 non-null  int64
20   Property Location                   29644 non-null  object
21   Co-Applicant                       30000 non-null  int64
22   Property Price                      30000 non-null  float64
23   Loan Sanction Amount (USD)          29660 non-null  float64
dtypes: float64(8), int64(5), object(11)
memory usage: 5.5+ MB
None
```

```
print(df.columns)
```

```
Index(['Customer ID', 'Name', 'Gender', 'Age', 'Income (USD)',
      'Income Stability', 'Profession', 'Type of Employment', 'Location',
      'Loan Amount Request (USD)', 'Current Loan Expenses (USD)',
      'Expense Type 1', 'Expense Type 2', 'Dependents', 'Credit Score',
      'No. of Defaults', 'Has Active Credit Card', 'Property ID',
      'Property Age', 'Property Type', 'Property Location', 'Co-Applicant',
      'Property Price', 'Loan Sanction Amount (USD)'],
      dtype='object')
```

```
print(df.head())
```

<bound method NDFrame.head of					Customer ID	Name	Gender	Age	Income (USD)	\
0	C-36995	Frederica	Shealy	F	56	1933.05				
1	C-33999	America	Calderone	M	32	4952.91				
2	C-3770	Rosetta	Verne	F	65	988.19				
3	C-26480	Zoe	Chitty	F	65	NaN				
4	C-23459	Afton	Venema	F	31	2614.77				
...	...	...	...	...	...	...				
29995	C-43723	Angelyn	Clevenger	M	38	4969.41				
29996	C-32511	Silas	Slaugh	M	20	1606.88				
29997	C-5192	Carmelo	Lone	F	49	NaN				
29998	C-12172	Carolann	Osby	M	38	2417.71				
29999	C-33003	Bridget	Garibaldi	F	63	3068.24				
Income Stability					Profession	Type of Employment	\			
0		Low			Working	Sales staff				
1		Low			Working	NaN				
2		High			Pensioner	NaN				
3		High			Pensioner	NaN				

```
df.isnull().sum()
```

```
Customer ID      0
Name             0
Gender          53
Age             0
Income (USD)     4576
Income Stability 1683
Profession       0
Type of Employment 7270
Location         0
Loan Amount Request (USD) 0
Current Loan Expenses (USD) 172
Expense Type 1   0
Expense Type 2   0
Dependents      2493
Credit Score    1703
No. of Defaults  0
Has Active Credit Card 1566
Property ID      0
Property Age     4850
Property Type    0
Property Location 356
Co-Applicant     0
Property Price   0
Loan Sanction Amount (USD) 340
dtype: int64
```

```
#dropping unessential columns
```

```
df=df.drop("Customer ID",axis=1)
```

```

df=df.drop("Name",axis=1)

# Handling missing values
for col in df.columns:
    if df[col].dtype=='object':
        df[col]=df[col].fillna(df[col].mode()[0])
    else:
        df[col]=df[col].fillna(df[col].median())

#one hot encoding
df=pd.get_dummies(df,drop_first=True)

#Outlier Handling
df = df[df["Income (USD)"] < df["Income (USD)"].quantile(0.99)]
df = df[df["Property Price"] < df["Property Price"].quantile(0.99)]

median=df.loc[df["Current Loan Expenses (USD)"] >=0,"Current Loan Expenses (USD)"].median()
df.loc[df["Current Loan Expenses (USD)"]<0,"Current Loan Expenses (USD)"]=median

df['Loan Sanction Amount (USD)']=df['Loan Sanction Amount (USD)'].fillna(df['Loan Sanction Amount (USD)'].median())
(df["Loan Sanction Amount (USD)"]<0).sum()

median=df.loc[df["Loan Sanction Amount (USD)"] >=0,"Loan Sanction Amount (USD)"].median()
df.loc[df["Loan Sanction Amount (USD)"]<0,"Loan Sanction Amount (USD)"]=median

#Log transformation
df["log_income"] = np.log1p(df["Income (USD)"])
df["log_expense"] = np.log1p(df["Current Loan Expenses (USD)"])
df['log_loan_amount'] = np.log1p(df['Loan Sanction Amount (USD)'])

#Adding Ratio-Based Features
df["Loan_to_Income"] = df["Loan Amount Request (USD)"] / (df["Income (USD)"] + 1)

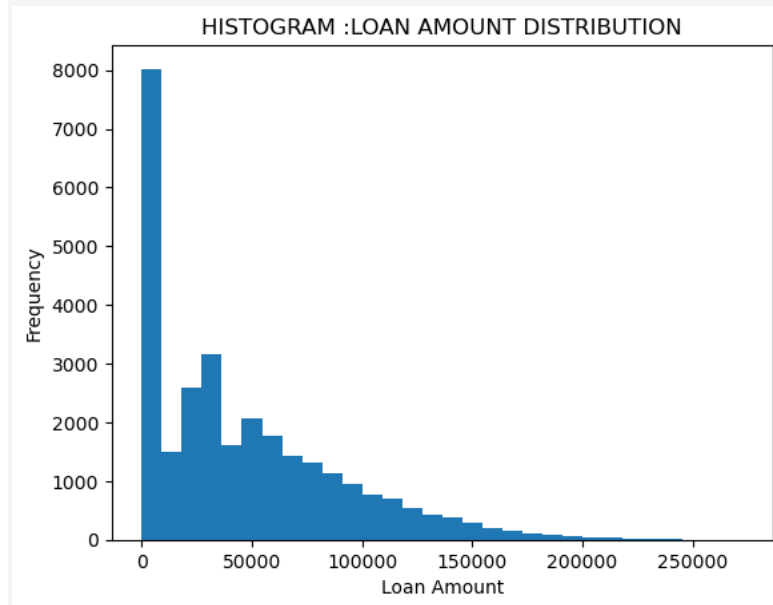
#selecting features(x,y)
x=df.drop(["Loan Sanction Amount (USD)"],axis=1)
y=df["Loan Sanction Amount (USD)"]
print(df.select_dtypes(include='object').columns)

#Normalizing or standardizing the features
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
x_scaled=scaler.fit_transform(x)

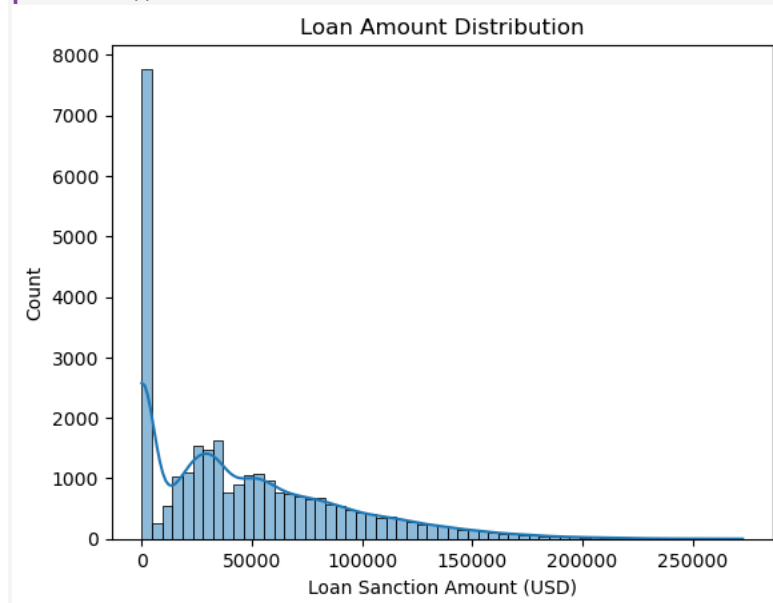
import matplotlib.pyplot as plt

```

```
import seaborn as sns
plt.hist(y,bins=30)
plt.title("HISTOGRAM :LOAN AMOUNT DISTRIBUTION")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.show()
```

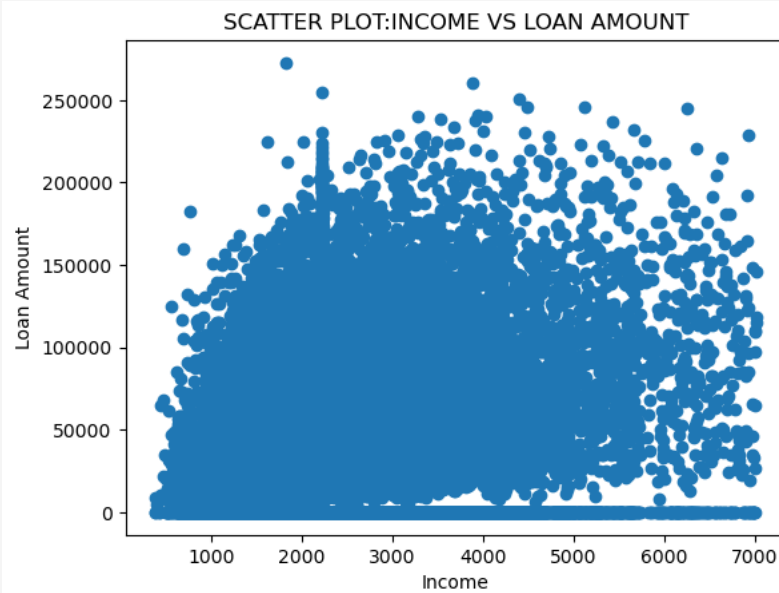


```
sns.histplot(df["Loan Sanction Amount (USD)"],kde="true")
plt.title("Loan Amount Distribution")
plt.show()
```

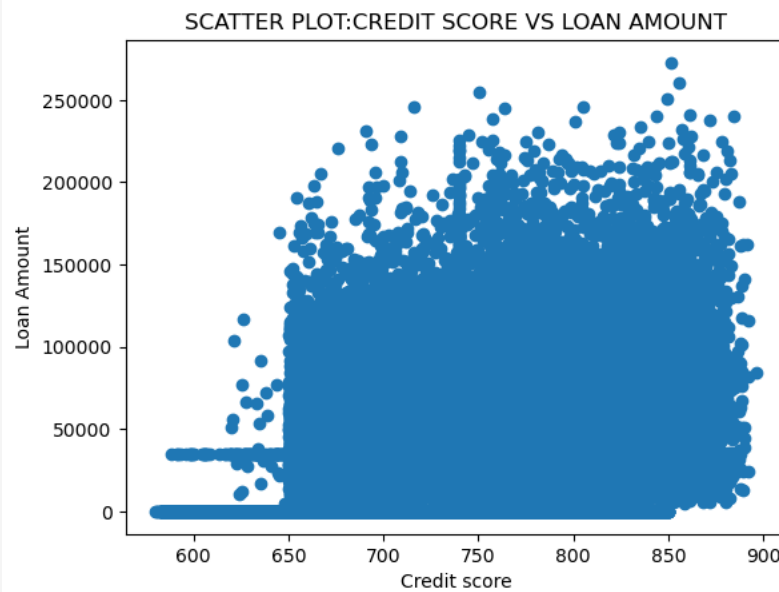


```
plt.scatter(df["Income (USD)"],df["Loan Sanction Amount (USD)"])
plt.title("SCATTER PLOT:INCOME VS LOAN AMOUNT")
plt.xlabel("Income")
```

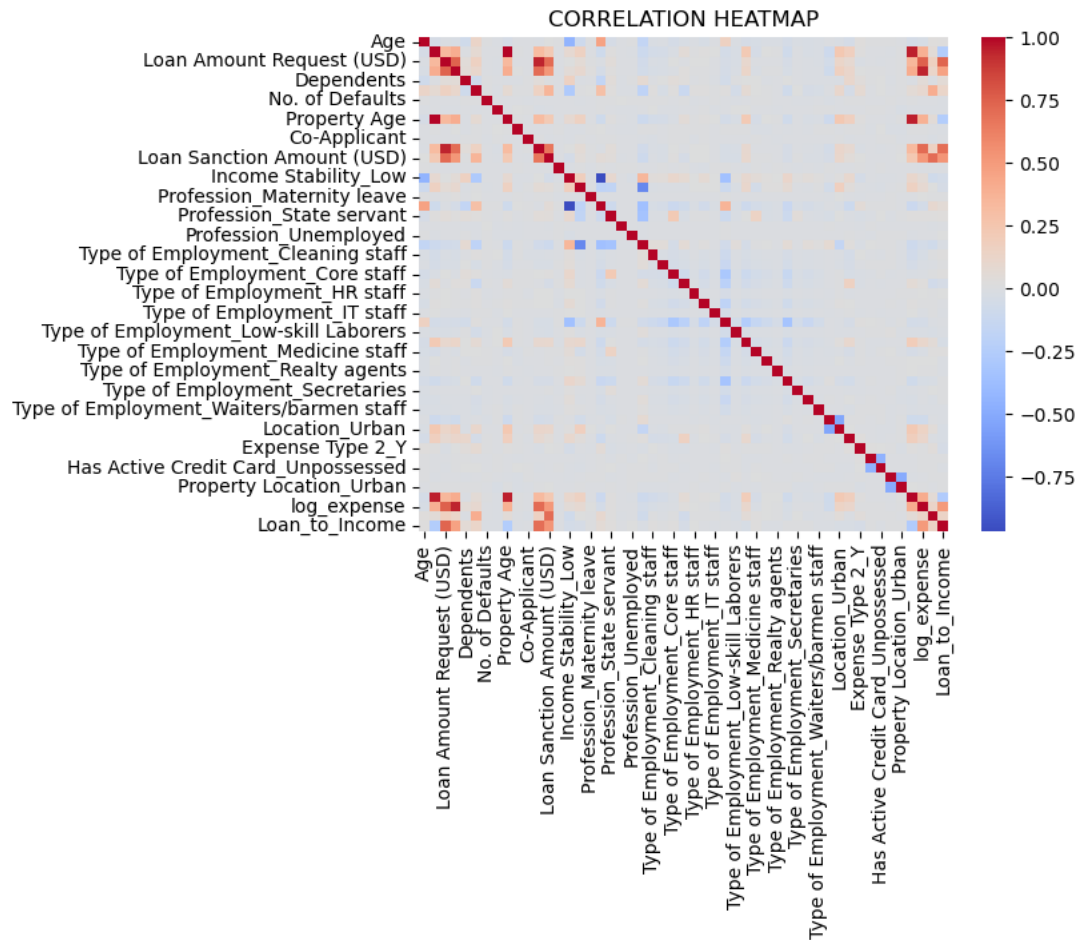
```
plt.ylabel("Loan Amount")
plt.show()
```



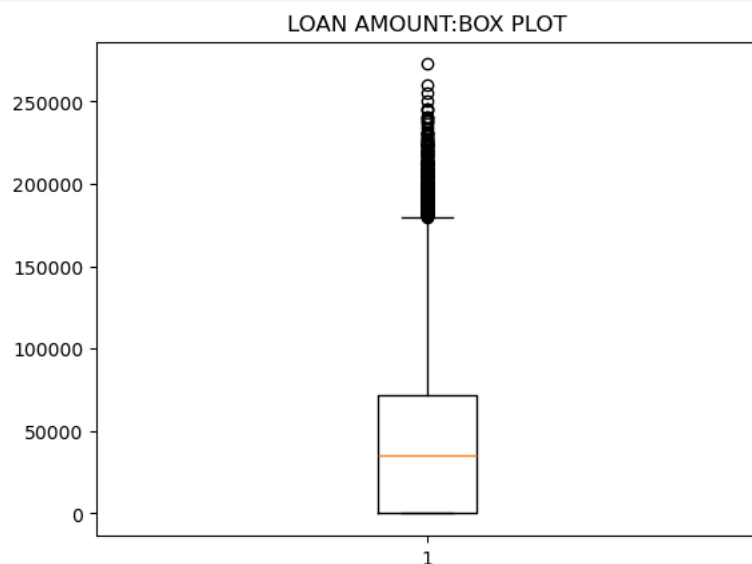
```
plt.scatter(df["Credit Score"],y)
plt.title("SCATTER PLOT:CREDIT SCORE VS LOAN AMOUNT")
plt.xlabel("Credit score")
plt.ylabel("Loan Amount")
plt.show()
```



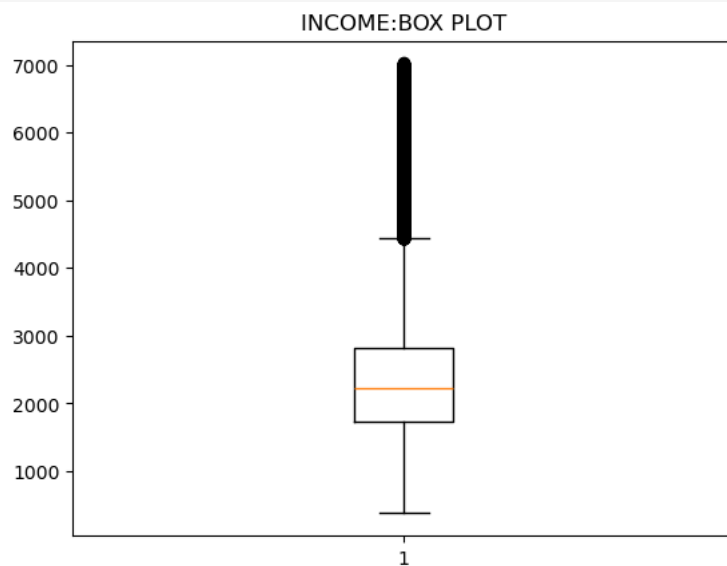
```
sns.heatmap(df.corr(),cmap="coolwarm")
plt.title("CORRELATION HEATMAP")
plt.show()
```



```
plt.boxplot(y)
plt.title("LOAN AMOUNT:BOX PLOT")
plt.show()
```



```
plt.boxplot(df["Income (USD)"])
plt.title("INCOME:BOX PLOT")
plt.show()
```



```
#model training
from sklearn.linear_model import LinearRegression
lr=LinearRegression()
lr.fit(x_scaled,y)
print("Intercept(b)",lr.intercept_)
print("Coefficient(w)",lr.coef_)

#splitting data
from sklearn.model_selection import train_test_split
x_train,x_temp,y_train,y_temp=train_test_split(x_scaled,y,test_size=0.3,random_state=42)
x_val,x_test,y_val,y_test=train_test_split(x_temp,y_temp,test_size=0.5,random_state=42)

#model training
from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)

#predicting
y_test_pred=model.predict(x_test)
y_val_pred=model.predict(x_val)

#model evaluation
from sklearn.metrics import mean_squared_error,mean_absolute_error,r2_score
mse=mean_squared_error(y_test,y_test_pred)
mae=mean_absolute_error(y_test,y_test_pred)
```



```

r2=r2_score(y_test,y_test_pred)
import numpy as np
rmse=np.sqrt(mse)
print("PERFORMANCE")
print("Mean Sqaure Error:",mse)
print("Mean Absolute Error:",mae)
print("R-Mean Sqaure Error:",rmse)
print("R2 SCORE:",r2)

```

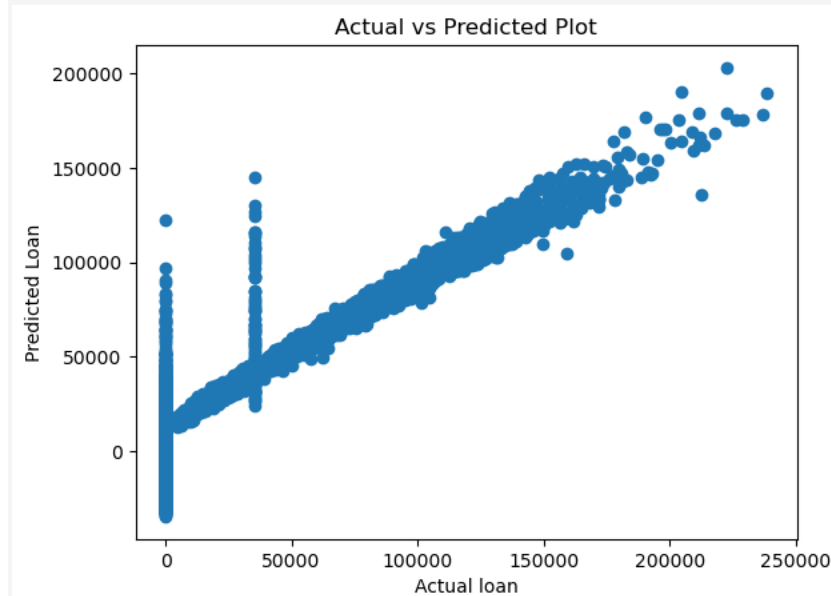
PERFORMANCE

Mean Sqaure Error: 277619656.2536446
 Mean Absolute Error: 11675.279183134779
 R-Mean Sqaure Error: 16661.922345685223
 R2 SCORE: 0.8608259525754163

```

#VISUALIZATION
#Actual vs Predicted Plot:
plt.scatter(y_test,y_test_pred)
plt.title("Actual vs Predicted Plot")
plt.xlabel("Actual loan")
plt.ylabel("Predicted Loan")
plt.show()

```

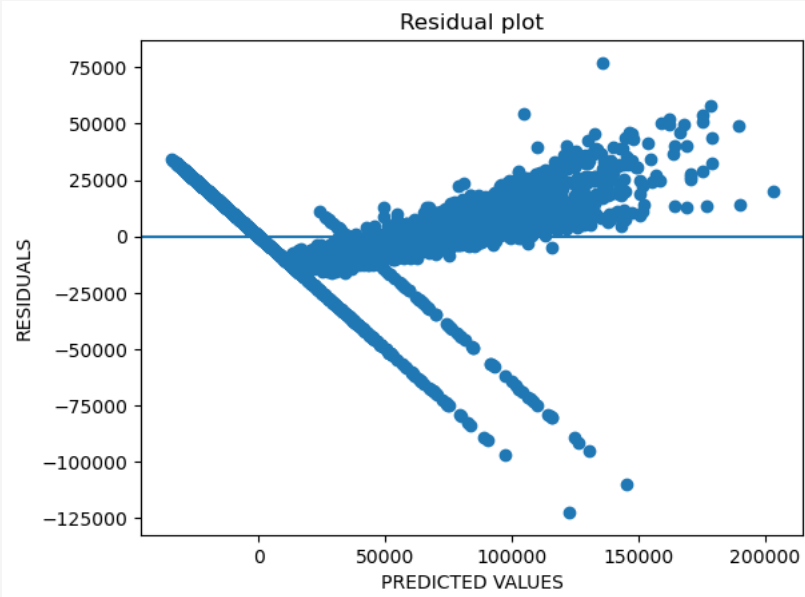


```

#Residual Plot
residuals=y_test-y_test_pred
plt.axhline(0)
plt.scatter(y_test_pred,residuals)
plt.xlabel("PREDICTED VALUES")
plt.ylabel("RESIDUALS")
plt.title("Residual plot")

```

```
plt.show()
```



```
#Bar Plot of Feature Coefficients
```

```
features=x.columns
```

```
coefficient=model.coef_
```

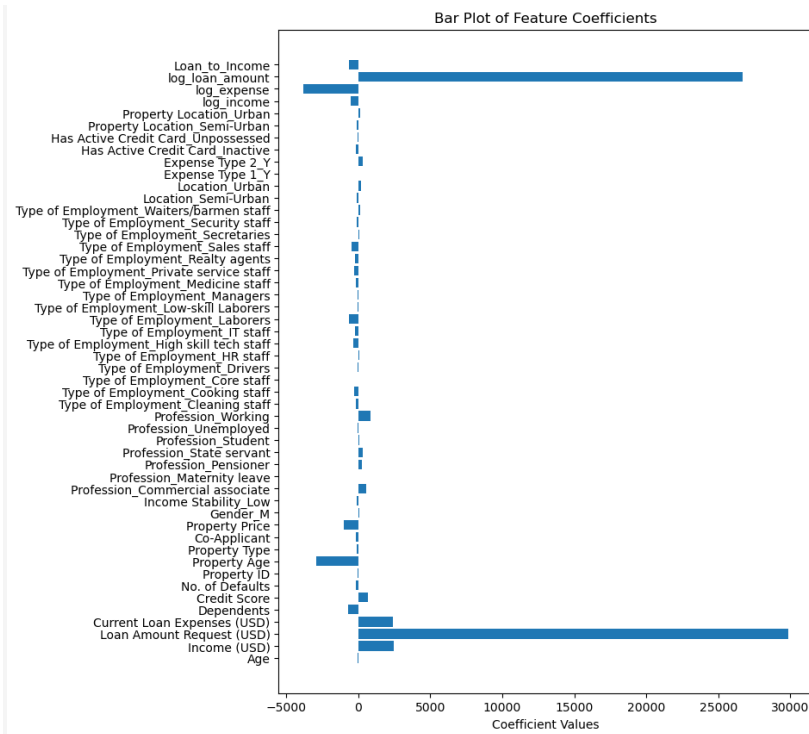
```
plt.figure(figsize=(8,10))
```

```
plt.barh(features,coefficient)
```

```
plt.title("Bar Plot of Feature Coefficients")
```

```
plt.xlabel("Coefficient Values")
```

```
plt.show()
```



```
# K-Fold Cross-Validation
from sklearn.model_selection import KFold
kf=KFold(n_splits=5,shuffle=True,random_state=42)
fold=1
mae_list,mse_list,rmse_list,r2_list=[],[],[],[]

for train,test in kf.split(x_scaled):
    train_x,test_x=x_scaled[train],x_scaled[test]
    train_y,test_y=y.iloc[train],y.iloc[test]

    lr=LinearRegression()
    lr.fit(train_x,train_y)

    y_pred=lr.predict(test_x)

    mae = mean_absolute_error(test_y, y_pred)
    mse = mean_squared_error(test_y, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(test_y, y_pred)
    print(f"Fold {fold} : MAE:{mae:.2f}, MSE:{mse:.2f}, RMSE:{rmse:.2f}, R2:{r2:.2f}")

    mae_list.append(mae)
    mse_list.append(mse)
    rmse_list.append(rmse)
    r2_list.append(r2)

    fold += 1
```

```

print("\nAVERAGE CROSS VALIDATION METRICS ")
print("Average MAE :", np.mean(mae_list))
print("Average MSE :", np.mean(mse_list))
print("Average RMSE:", np.mean(rmse_list))
print("Average R2 :", np.mean(r2_list))

```

```

Fold 1 : MAE:11768.24, MSE:281030217.10, RMSE:16763.96, R2:0.86
Fold 2 : MAE:11713.23, MSE:279888702.58, RMSE:16729.87, R2:0.86
Fold 3 : MAE:11895.15, MSE:301148161.91, RMSE:17353.62, R2:0.85
Fold 4 : MAE:11660.95, MSE:281598689.82, RMSE:16780.90, R2:0.86
Fold 5 : MAE:11471.74, MSE:281794235.39, RMSE:16786.73, R2:0.85

```

```

AVERAGE CROSS VALIDATION METRICS
Average MAE : 11701.861892980978
Average MSE : 285092001.35970193
Average RMSE: 16883.01638670866
Average R2 : 0.8575086419181647

```

```

#SPLITTING INTO DIFFERENT RATIOS
splits = [(0.7,0.3), (0.6,0.4), (0.75,0.25), (0.80,0.20)]

for tr_ratio, te_ratio in splits:
    X_train, X_test, y_train, y_test = train_test_split(
        x_scaled, y, test_size=te_ratio, random_state=42
    )

    lr = LinearRegression()
    lr.fit(X_train, y_train)

    print("SPLIT:", int(tr_ratio*100), "/", int(te_ratio*100))
    print("Train size:", len(y_train))
    print("Test size :", len(y_test))
    r2 = r2_score(y_test, lr.predict(X_test))
    N = X_test.shape[0] + X_train.shape[0]
    p = X_train.shape[1]
    adjusted_r2 = 1 - (1 - r2) * (N - 1) / (N - p - 1)
    print("R2:", r2)
    print("Adjusted R2:", adjusted_r2)
    print("MAE:", mean_absolute_error(y_test, lr.predict(X_test)))
    print("MSE:", mean_squared_error(y_test, lr.predict(X_test)))
    print("-----")

```

```

Split: 70 / 30
Train size: 20582
Test size : 8821

```

R2: 0.8583985696898294
Adjusted R2: 0.8581573571143487
MAE: 11785.840471977262
MSE: 284703401.7086592

Split: 60 / 40
Train size: 17641
Test size : 11762
R2: 0.8618249243542235
Adjusted R2: 0.8615895484417716
MAE: 11808.636007545398
MSE: 281162416.4426841

Split: 75 / 25
Train size: 22052
Test size : 7351
R2: 0.8586555939028726
Adjusted R2: 0.8584148191582264
MAE: 11778.318174896674
MSE: 285134125.647867

Split: 80 / 20
Train size: 23522
Test size : 5881
R2: 0.8623867632619528
Adjusted R2: 0.8621523444204122
MAE: 11768.235383748233
MSE: 281030217.09844375

Table 1: Cross-Validation Results (K =5)

Fold	MAE	MSE	RM RMSE	R ² Score
Fold 1	11768.24	281030217.10	16763.96	0.86
Fold 2	11713.23	279888702.58	16729.87	0.86
Fold 3	11895.15	301148161.91	17353.62	0.85
Fold 4	11660.95	281598689.82	16780.90	0.86
Fold 5	11471.74	281794235.39	16786.73	0.85
Average	11701.86	285092001.36	16883.02	0.86

Table 2: Summary of Results for Loan Amount Prediction

Description	Student's Result
Dataset Size (after preprocessing)	29403 rows $\times$ 24 columns
Train/Test Split Ratio	70:30, 60:40, 75:25, 80:20
Feature(s) Used for Prediction	Total Income, Loan to Income Ratio, Expense to Income Ratio, Credit Score
Model Used	Linear Regression
Cross-Validation Used? (Yes/No)	Yes
If Yes, Number of Folds (K)	5
Reference to CV Results Table	Table 1
Mean Absolute Error (MAE) on Test Set	11785.84
Mean Squared Error (MSE) on Test Set	284703401.71
Root Mean Squared Error (RMSE) on Test Set	16883.02
R ² Score on Test Set	0.86
Adjusted R ² Score on Test Set	0.86
Most Influential Feature(s)	Total Income, Credit Score

Observations from Residual Plot	Residuals are randomly distributed around zero
Interpretation of Predicted vs Actual Plot	Predicted values follow actual values with minor deviation
Any Overfitting or Underfitting Observed?	No
Brief Justification	Training and test errors are close; no pattern in residuals

Best Practices

- Always convert categorical data to numerical form before scaling
- Use median/mode for missing values
- Inspect correlations before training
- Keep test data unseen
- Visualize residuals to verify linearity

Learning Outcomes

- Understood the mathematical foundation of Linear Regression
- Gained experience in data preprocessing and normalization
- Learned to interpret coefficients and residual plots
- Applied cross-validation for robust evaluation
- Visualized model performance effectively